PROJECT REPORT ;

- *The following project report aims to develop an efficient and accurate model to predict the likelihood of a patient having hypothyroid.*
- *This will be achieved by training classification models on the provided dataset, which contains labeled data indicating the presence or absence of hypothyroidism.*
- *The goal is to preprocess the data, analyze it for insights, and apply various classification algorithms to determine the best-performing model based on multiple evaluation metrics.*

In [259]:
```
#import the required libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

In [260]:
```
#to access the dataset
df = pd.read_csv("hypothyroid_classification.csv")
```

DATA EXPLORATION :

In [261]:
```
#to get informations about the dataset
display(df.head())
display(df.describe())
display(df.info())
```

	age	sex	on thyroxine	query on thyroxine	on antithyroid medication	sick	pregnant	thyroid surgery	I131 treatment	query hypothyroid	...	TT4 measured	TT4	T4U measured
0	41	F	f	f	f	f	f	f	f	f	...	t	125	t
1	23	F	f	f	f	f	f	f	f	f	...	t	102	f
2	46	M	f	f	f	f	f	f	f	f	...	t	109	t
3	70	F	t	f	f	f	f	f	f	f	...	t	175	f
4	70	F	f	f	f	f	f	f	f	f	...	t	61	t

5 rows × 30 columns

	age	sex	on thyroxine	query on thyroxine	on antithyroid medication	sick	pregnant	thyroid surgery	I131 treatment	query hypothyroid	...	TT4 measured	TT4
count	3772	3772	3772	3772	3772	3772	3772	3772	3772	3772	...	3772	3772
unique	94	3	2	2	2	2	2	2	2	2	...	2	242
top	59	F	f	f	f	f	f	f	f	f	...	t	?
freq	95	2480	3308	3722	3729	3625	3719	3719	3713	3538	...	3541	231

4 rows × 30 columns

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3772 entries, 0 to 3771
Data columns (total 30 columns):
 #   Column                     Non-Null Count  Dtype 
---  ------                     --------------  ----- 
 0   age                        3772 non-null   object
 1   sex                        3772 non-null   object
 2   on thyroxine               3772 non-null   object
 3   query on thyroxine         3772 non-null   object
 4   on antithyroid medication  3772 non-null   object
 5   sick                       3772 non-null   object
 6   pregnant                   3772 non-null   object
 7   thyroid surgery            3772 non-null   object
 8   I131 treatment             3772 non-null   object
 9   query hypothyroid          3772 non-null   object
 10  query hyperthyroid         3772 non-null   object
 11  lithium                    3772 non-null   object
 12  goitre                     3772 non-null   object
 13  tumor                      3772 non-null   object
 14  hypopituitary              3772 non-null   object
 15  psych                      3772 non-null   object
 16  TSH measured               3772 non-null   object
 17  TSH                        3772 non-null   object
 18  T3 measured                3772 non-null   object
 19  T3                         3772 non-null   object
 20  TT4 measured               3772 non-null   object
 21  TT4                        3772 non-null   object
 22  T4U measured               3772 non-null   object
 23  T4U                        3772 non-null   object
 24  FTI measured               3772 non-null   object
 25  FTI                        3772 non-null   object
 26  TBG measured               3772 non-null   object
 27  TBG                        3772 non-null   object
 28  referral source            3772 non-null   object
 29  binaryClass                3772 non-null   object
dtypes: object(30)
memory usage: 884.2+ KB
None
```

DATA PROCESSING AND CLEANING :

In [262]:
```
# Replace '?' with NaN
df.replace('?', np.nan, inplace=True)
```

```
C:\Users\hp\AppData\Local\Temp\ipykernel_20624\4084823351.py:2: FutureWarning: Downcasting behavior in
`replace` is deprecated and will be removed in a future version. To retain the old behavior, explicitly
call `result.infer_objects(copy=False)`. To opt-in to the future behavior, set `pd.set_option('future.n
o_silent_downcasting', True)`
  df.replace('?', np.nan, inplace=True)
```

In [263]:
```
# Define columns in the dataset
numerical_columns = ['age', 'TSH', 'T3', 'TT4', 'T4U', 'FTI']

categorical_columns = [
    'on thyroxine', 'query on thyroxine', 'on antithyroid medication', 'sick',
    'pregnant', 'thyroid surgery', 'I131 treatment', 'query hypothyroid',
    'query hyperthyroid', 'lithium', 'goitre', 'tumor', 'hypopituitary', 'psych',
    'TSH measured', 'T3 measured', 'TT4 measured', 'T4U measured', 'FTI measured',
    'TBG measured','referral source','sex']
```

In [264]:
```
# Convert numerical columns to float, coerce errors to NaN
for col in numerical_columns:
    df[col] = pd.to_numeric(df[col], errors='coerce')
```

In [265]:
```
# to change all the f and t value to 1 and 0
boolean_columns = [
    'on thyroxine', 'query on thyroxine', 'on antithyroid medication', 'sick',
    'pregnant', 'thyroid surgery', 'I131 treatment', 'query hypothyroid',
    'query hyperthyroid', 'lithium', 'goitre', 'tumor', 'hypopituitary', 'psych',
    'TSH measured', 'T3 measured', 'TT4 measured', 'T4U measured', 'FTI measured',
    'TBG measured']

# Convert boolean columns to boolean type
for col in boolean_columns:
    df[col] = df[col].map({'t': 1, 'f': 0})
```

In [266]:
```
print(df.head())
```

```
    age sex  on thyroxine  query on thyroxine  on antithyroid medication  \
0  41.0   F             0                   0                          0   
1  23.0   F             0                   0                          0   
2  46.0   M             0                   0                          0   
3  70.0   F             1                   0                          0   
4  70.0   F             0                   0                          0   

   sick  pregnant  thyroid surgery  I131 treatment  query hypothyroid  ...  \
0     0         0                0               0                  0  ...   
1     0         0                0               0                  0  ...   
2     0         0                0               0                  0  ...   
3     0         0                0               0                  0  ...   
4     0         0                0               0                  0  ...   

   TT4 measured    TT4  T4U measured   T4U  FTI measured    FTI  TBG measured  \
0             1  125.0             1  1.14             1  109.0             0   
1             1  102.0             0   NaN             0    NaN             0   
2             1  109.0             1  0.91             1  120.0             0   
3             1  175.0             0   NaN             0    NaN             0   
4             1   61.0             1  0.87             1   70.0             0   

   TBG  referral source  binaryClass  
0  NaN             SVHC            P  
1  NaN            other            P  
2  NaN            other            P  
3  NaN            other            P  
4  NaN              SVI            P  

[5 rows x 30 columns]
```

In [267]:
```
 #for the binaryClass column

print(f"\nUnique values in binaryClass before mapping: {df['binaryClass'].unique()}")
print(f"\nNull values in binaryClass before mapping: {df['binaryClass'].isnull().sum()}")

df['binaryClass'] = df['binaryClass'].map({'P': 1, 'N': 0})

print(f"\nUnique values in binaryClass after mapping: {df['binaryClass'].unique()}")
df['binaryClass'] = df['binaryClass'].astype('Int64')
print(f"\nUnique values in binaryClass after conversion: {df['binaryClass'].unique()}")
```

```
Unique values in binaryClass before mapping: ['P' 'N']

Null values in binaryClass before mapping: 0

Unique values in binaryClass after mapping: [1 0]

Unique values in binaryClass after conversion: <IntegerArray>
[1, 0]
Length: 2, dtype: Int64
```

In [268]:
```
display(df.head())
display(df.info())
```

	age	sex	on thyroxine	query on thyroxine	on antithyroid medication	sick	pregnant	thyroid surgery	I131 treatment	query hypothyroid	...	TT4 measured	TT4
0	41.0	F	0	0	0	0	0	0	0	0	...	1	125.0
1	23.0	F	0	0	0	0	0	0	0	0	...	1	102.0
2	46.0	M	0	0	0	0	0	0	0	0	...	1	109.0
3	70.0	F	1	0	0	0	0	0	0	0	...	1	175.0
4	70.0	F	0	0	0	0	0	0	0	0	...	1	61.0

5 rows × 30 columns

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3772 entries, 0 to 3771
Data columns (total 30 columns):
 #   Column                     Non-Null Count  Dtype  
---  ------                     --------------  -----  
 0   age                        3771 non-null   float64
 1   sex                        3622 non-null   object 
 2   on thyroxine               3772 non-null   int64  
 3   query on thyroxine         3772 non-null   int64  
 4   on antithyroid medication  3772 non-null   int64  
 5   sick                       3772 non-null   int64  
 6   pregnant                   3772 non-null   int64  
 7   thyroid surgery            3772 non-null   int64  
 8   I131 treatment             3772 non-null   int64  
 9   query hypothyroid          3772 non-null   int64  
 10  query hyperthyroid         3772 non-null   int64  
 11  lithium                    3772 non-null   int64  
 12  goitre                     3772 non-null   int64  
 13  tumor                      3772 non-null   int64  
 14  hypopituitary              3772 non-null   int64  
 15  psych                      3772 non-null   int64  
 16  TSH measured               3772 non-null   int64  
 17  TSH                        3403 non-null   float64
 18  T3 measured                3772 non-null   int64  
 19  T3                         3003 non-null   float64
 20  TT4 measured               3772 non-null   int64  
 21  TT4                        3541 non-null   float64
 22  T4U measured               3772 non-null   int64  
 23  T4U                        3385 non-null   float64
 24  FTI measured               3772 non-null   int64  
 25  FTI                        3387 non-null   float64
 26  TBG measured               3772 non-null   int64  
 27  TBG                        0 non-null      float64
 28  referral source            3772 non-null   object 
 29  binaryClass                3772 non-null   Int64  
dtypes: Int64(1), float64(7), int64(20), object(2)
memory usage: 887.9+ KB
None
```

MISSING VALUE IDENTIFICATION AND TREATMENT

IDENTIFICATION :

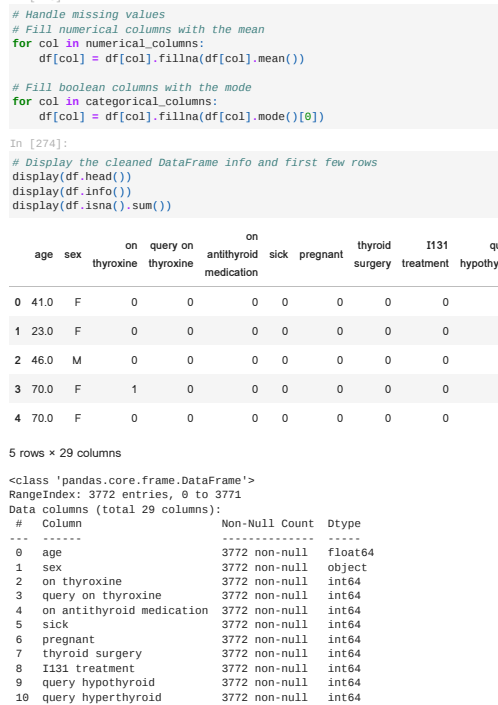
In [269]:
```
#To check the missing values
print("THE NUMBER OF MISSING VALUE IN EACH COLUMN:\n")
display(df.isna().sum())
```

```
THE NUMBER OF MISSING VALUE IN EACH COLUMN:

age                             1
sex                           150
on thyroxine                    0
query on thyroxine              0
on antithyroid medication       0
sick                            0
pregnant                        0
thyroid surgery                 0
I131 treatment                  0
query hypothyroid               0
query hyperthyroid              0
lithium                         0
goitre                          0
tumor                           0
hypopituitary                   0
psych                           0
TSH measured                    0
TSH                           369
T3 measured                     0
T3                            769
TT4 measured                    0
TT4                           231
T4U measured                    0
T4U                           387
FTI measured                    0
FTI                           385
TBG measured                    0
TBG                          3772
referral source                 0
binaryClass                     0
dtype: int64
```

VISUALISATION OF MISSING VALUES

In [270]:
```
# Visualize missing values
sns.heatmap(df.isnull(), cbar=False, cmap='viridis')
plt.title('Missing Values Heatmap')
plt.show()
```

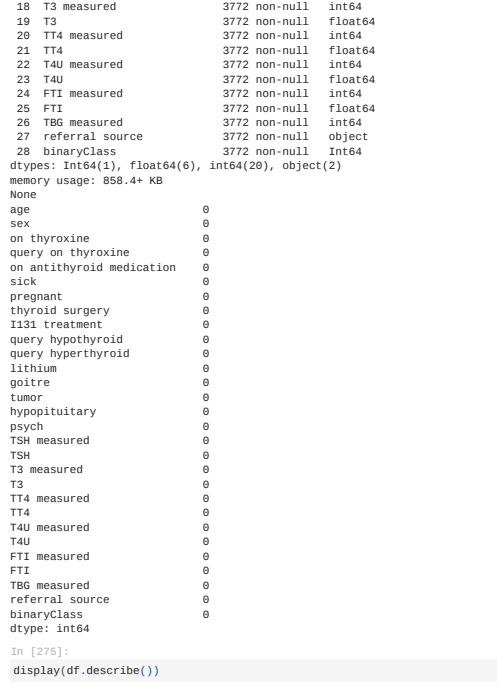

Missing Values Heatmap

MISSING VALUE TREATMENT :

1. DROPPING THE COLUMN

In [271]:
```
# Drop columns with all values missing
df.dropna(axis=1, how='all', inplace=True)
```

In [272]:
```
# Visualize missing values
sns.heatmap(df.isnull(), cbar=False, cmap='viridis')
plt.title('Missing Values Heatmap')
plt.show()
```


Missing Values Heatmap

2. USING MEAN/MODE METHOD

In [273]:
```
# Handle missing values
# Fill numerical columns with the mean
for col in numerical_columns:
    df[col] = df[col].fillna(df[col].mean())

# Fill boolean columns with the mode
for col in categorical_columns:
    df[col] = df[col].fillna(df[col].mode()[0])
```

In [274]:
```
# Display the cleaned DataFrame info and first few rows
display(df.head())
display(df.info())
display(df.isna().sum())
```

	age	sex	on thyroxine	query on thyroxine	on antithyroid medication	sick	pregnant	thyroid surgery	I131 treatment	query hypothyroid	...	T3	TT4 measured	TT4
0	41.0	F	0	0	0	0	0	0	0	0	...	2.5000	1	125.0
1	23.0	F	0	0	0	0	0	0	0	0	...	2.0000	1	102.0
2	46.0	M	0	0	0	0	0	0	0	0	...	2.0135	1	109.0
3	70.0	F	1	0	0	0	0	0	0	0	...	1.9000	1	175.0
4	70.0	F	0	0	0	0	0	0	0	0	...	1.2000	1	61.0

5 rows × 29 columns

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3772 entries, 0 to 3771
Data columns (total 29 columns):
 #   Column                     Non-Null Count  Dtype  
---  ------                     --------------  -----  
 0   age                        3772 non-null   float64
 1   sex                        3772 non-null   object 
 2   on thyroxine               3772 non-null   int64  
 3   query on thyroxine         3772 non-null   int64  
 4   on antithyroid medication  3772 non-null   int64  
 5   sick                       3772 non-null   int64  
 6   pregnant                   3772 non-null   int64  
 7   thyroid surgery            3772 non-null   int64  
 8   I131 treatment             3772 non-null   int64  
 9   query hypothyroid          3772 non-null   int64  
 10  query hyperthyroid         3772 non-null   int64  
 11  lithium                    3772 non-null   int64  
 12  goitre                     3772 non-null   int64  
 13  tumor                      3772 non-null   int64  
 14  hypopituitary              3772 non-null   int64  
 15  psych                      3772 non-null   int64  
 16  TSH measured               3772 non-null   int64  
 17  TSH                        3772 non-null   float64
 18  T3 measured                3772 non-null   int64  
 19  T3                         3772 non-null   float64
 20  TT4 measured               3772 non-null   int64  
 21  TT4                        3772 non-null   float64
 22  T4U measured               3772 non-null   int64  
 23  T4U                        3772 non-null   float64
 24  FTI measured               3772 non-null   int64  
 25  FTI                        3772 non-null   float64
 26  TBG measured               3772 non-null   int64  
 27  referral source            3772 non-null   object 
 28  binaryClass                3772 non-null   Int64  
dtypes: Int64(1), float64(6), int64(20), object(2)
memory usage: 858.4+ KB
None
age                          0
sex                          0
on thyroxine                 0
query on thyroxine           0
on antithyroid medication    0
sick                         0
pregnant                     0
thyroid surgery              0
I131 treatment               0
query hypothyroid            0
query hyperthyroid           0
lithium                      0
goitre                       0
tumor                        0
hypopituitary                0
psych                        0
TSH measured                 0
TSH                          0
T3 measured                  0
T3                           0
TT4 measured                 0
TT4                          0
T4U measured                 0
T4U                          0
FTI measured                 0
FTI                          0
TBG measured                 0
referral source              0
binaryClass                  0
dtype: int64
```

In [275]:
```
display(df.describe())
```

	age	on thyroxine	query on thyroxine	on antithyroid medication	sick	pregnant	thyroid surgery	I131 treatment
count	3772.000000	3772.000000	3772.000000	3772.000000	3772.000000	3772.000000	3772.000000	3772.000000
mean	51.735879	0.123012	0.013256	0.011400	0.038971	0.014051	0.014051	0.015642
std	20.082295	0.328494	0.114382	0.106174	0.193552	0.117716	0.117716	0.124101
min	1.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	36.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
50%	54.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
75%	67.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000
max	455.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000	1.000000

8 rows × 27 columns

VISUALISATION OF THE DATA

EXPLORATORY DATA ANALYSIS FOR THE NUMERICAL COLUMNS

In [276]:
```
from scipy.stats import zscore
from sklearn.preprocessing import RobustScaler, MinMaxScaler
from scipy.stats import boxcox
```

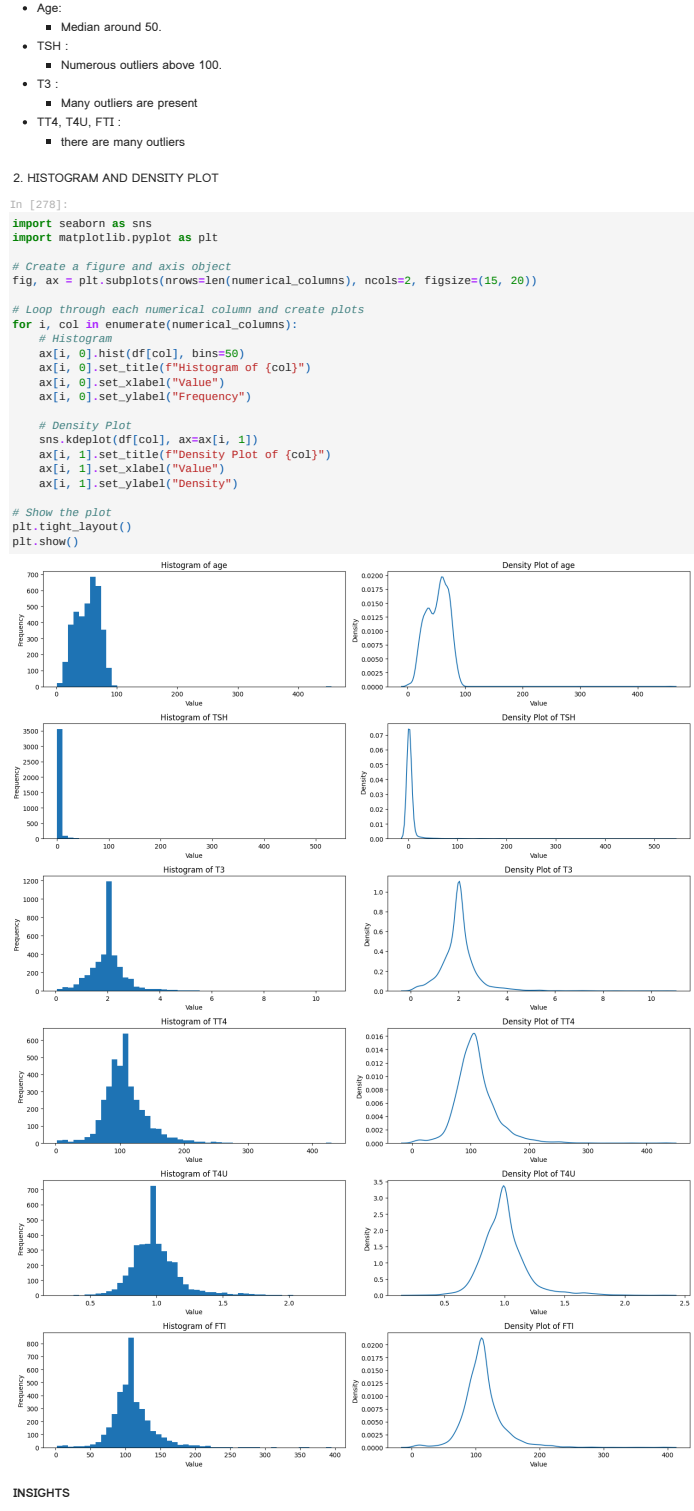
1. BOXPLOT

In [277]:
```
import seaborn as sns
import matplotlib.pyplot as plt

# Create a figure and axis object
fig, ax = plt.subplots(nrows=len(numerical_columns), ncols=1, figsize=(10, 20))

# Loop through each numerical column and create a boxplot
for i, col in enumerate(numerical_columns):
    sns.boxplot(x=df[col], ax=ax[i])
    ax[i].set_title(f"Boxplot of {col}")
    ax[i].set_xlabel("")
    ax[i].set_ylabel("Value")

# Show the plot
plt.tight_layout()
plt.show()
```



INSIGHTS

- Age:
    - Median around 50.
- TSH :
    - Numerous outliers above 100.
- T3 :
    - Many outliers are present
- TT4, T4U, FTI :
    - there are many outliers

2. HISTOGRAM AND DENSITY PLOT

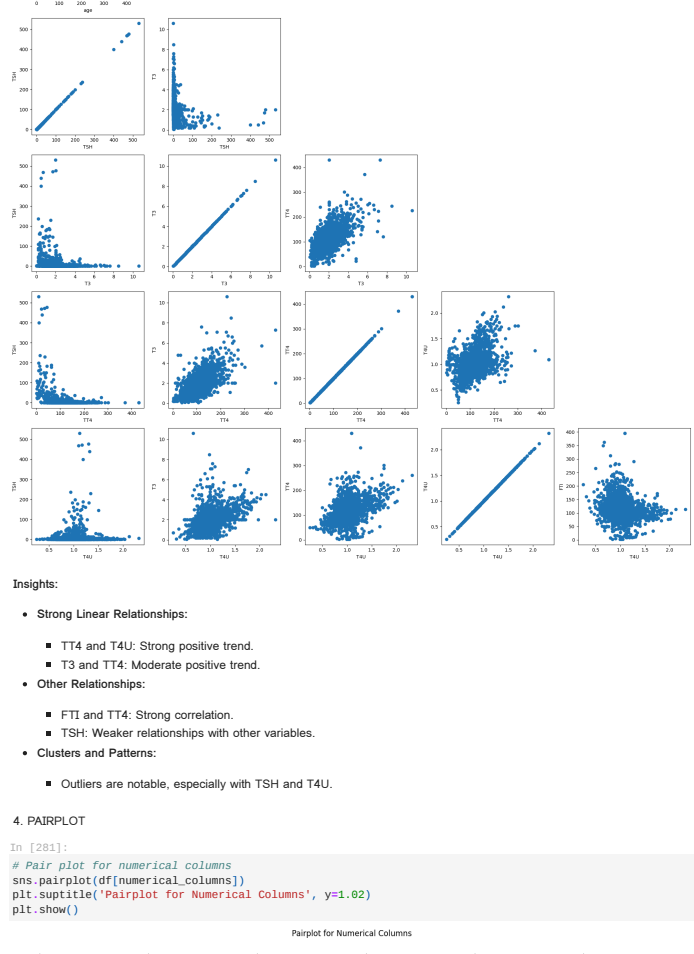
In [278]:
```
import seaborn as sns
import matplotlib.pyplot as plt

# Create a figure and axis object
fig, ax = plt.subplots(nrows=len(numerical_columns), ncols=2, figsize=(15, 20))

# Loop through each numerical column and create plots
for i, col in enumerate(numerical_columns):
    # Histogram
    ax[i, 0].hist(df[col], bins=50)
    ax[i, 0].set_title(f"Histogram of {col}")
    ax[i, 0].set_xlabel("Value")
    ax[i, 0].set_ylabel("Frequency")

    # Density Plot
    sns.kdeplot(df[col], ax=ax[i, 1])
    ax[i, 1].set_title(f"Density Plot of {col}")
    ax[i, 1].set_xlabel("Value")
    ax[i, 1].set_ylabel("Density")

# Show the plot
plt.tight_layout()
plt.show()
```



INSIGHTS

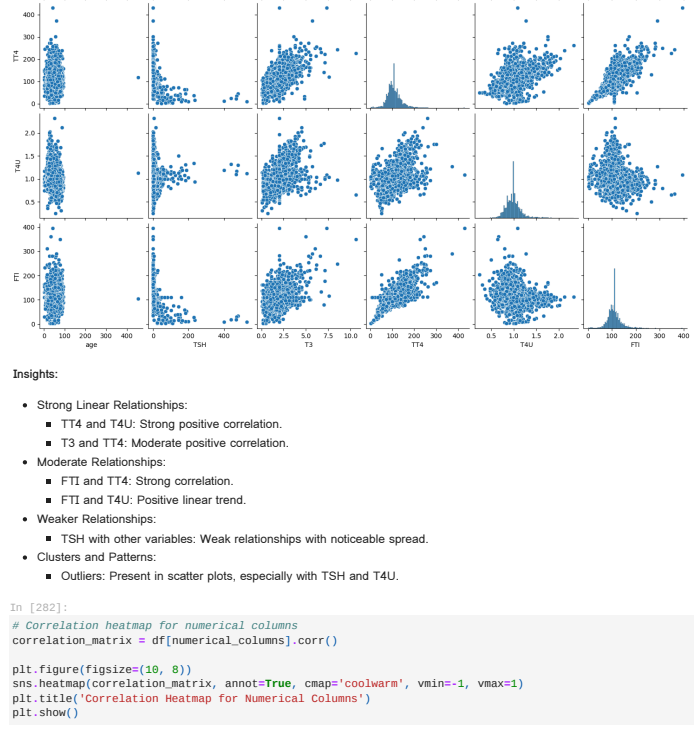
- **Age:** Roughly normal, potential outliers at the higher end.
- **TSH:** Highly skewed with extreme outliers.
- **T3:** Roughly normal with potential outliers.
- **TT4:** Slightly skewed with potential outliers.
- **T4U:** Mostly normal but with some outliers.
- **FTI:** Slightly skewed with potential outliers.

In [279]:
```
##### 3. SCATTERPLOT
```

In [280]:
```
# Scatter plots for numerical columns against each other
fig, ax = plt.subplots(nrows=len(numerical_columns)-1, ncols=len(numerical_columns)-1, figsize=(20, 20))

for i, col1 in enumerate(numerical_columns[:-1]):
    for j, col2 in enumerate(numerical_columns[1:]):
        if i >= j:
            ax[i, j].scatter(df[col1], df[col2])
            ax[i, j].set_xlabel(col1)
            ax[i, j].set_ylabel(col2)
        else:
            ax[i, j].axis('off')

plt.tight_layout()
plt.show()
```

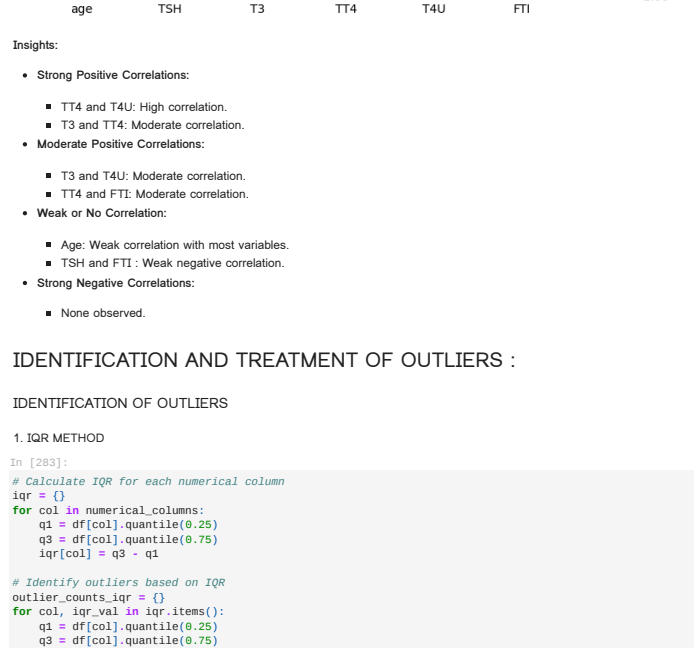


Insights:

- **Strong Linear Relationships:**
    - TT4 and T4U: Strong positive trend.
    - T3 and TT4: Moderate positive trend.
- **Other Relationships:**
    - FTI and TT4: Strong correlation.
    - TSH: Weaker relationships with other variables.
- **Clusters and Patterns:**
    - Outliers are notable, especially with TSH and T4U.

4. PAIRPLOT

In [281]:
```
# Pair plot for numerical columns
sns.pairplot(df[numerical_columns])
plt.suptitle('Pairplot for Numerical Columns', y=1.02)
plt.show()
```

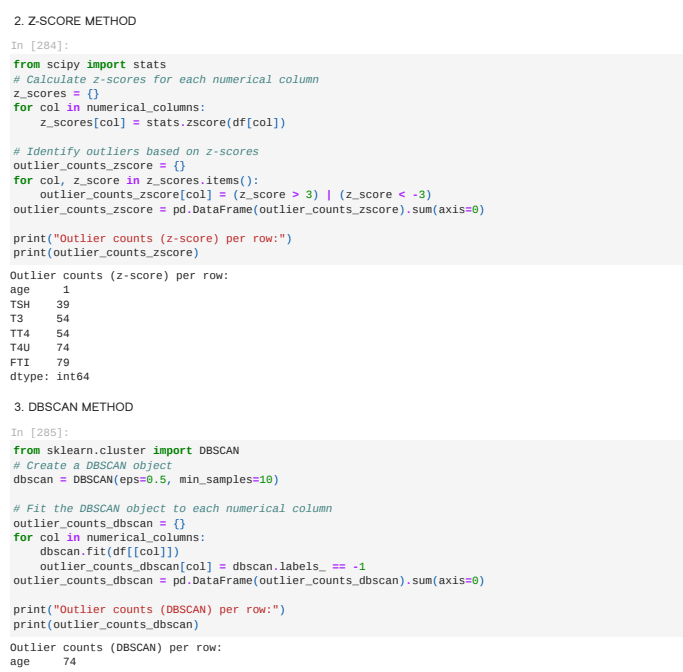

Pairplot for Numerical Columns

Insights:

- Strong Linear Relationships:
    - TT4 and T4U: Strong positive correlation.
    - T3 and TT4: Moderate positive correlation.
- Moderate Relationships:
    - FTI and TT4: Strong correlation.
    - FTI and T4U: Positive linear trend.
- Weaker Relationships:
    - TSH with other variables: Weak relationships with noticeable spread.
- Clusters and Patterns:
    - Outliers: Present in scatter plots, especially with TSH and T4U.

In [282]:
```
# Correlation heatmap for numerical columns
correlation_matrix = df[numerical_columns].corr()

plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title('Correlation Heatmap for Numerical Columns')
plt.show()
```


Correlation Heatmap for Numerical Columns

Insights:

- **Strong Positive Correlations:**
    - TT4 and T4U: High correlation.
    - T3 and TT4: Moderate correlation.
- **Moderate Positive Correlations:**
    - T3 and T4U: Moderate correlation.
    - TT4 and FTI: Moderate correlation.
- **Weak or No Correlation:**
    - Age: Weak correlation with most variables.
    - TSH and FTI : Weak negative correlation.
- **Strong Negative Correlations:**
    - None observed.

IDENTIFICATION AND TREATMENT OF OUTLIERS :

IDENTIFICATION OF OUTLIERS

1. IQR METHOD

In [283]:
```
# Calculate IQR for each numerical column
iqr = {}
for col in numerical_columns:
    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)
    iqr[col] = q3 - q1

# Identify outliers based on IQR
outlier_counts_iqr = {}
for col, iqr_val in iqr.items():
    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)
    outlier_counts_iqr[col] = (df[col] < (q1 - 1.5 * iqr_val)) | (df[col] > (q3 + 1.5 * iqr_val))

outlier_counts_iqr = pd.DataFrame(outlier_counts_iqr).sum(axis=0)

print("Outlier counts (IQR) per row:")
print(outlier_counts_iqr)
```

```
Outlier counts (IQR) per row:
age      1
TSH    258
T3     456
TT4    215
T4U    205
FTI    277
dtype: int64
```

2. Z-SCORE METHOD

In [284]:
```
from scipy import stats
# Calculate z-scores for each numerical column
z_scores = {}
for col in numerical_columns:
    z_scores[col] = stats.zscore(df[col])

# Identify outliers based on z-scores
outlier_counts_zscore = {}
for col, z_score in z_scores.items():
    outlier_counts_zscore[col] = (z_score > 3) | (z_score < -3)
outlier_counts_zscore = pd.DataFrame(outlier_counts_zscore).sum(axis=0)

print("Outlier counts (z-score) per row:")
print(outlier_counts_zscore)
```

```
Outlier counts (z-score) per row:
age     1
TSH    39
T3     54
TT4    54
T4U    74
FTI    79
dtype: int64
```

3. DBSCAN METHOD

In [285]:
```
from sklearn.cluster import DBSCAN
# Create a DBSCAN object
dbscan = DBSCAN(eps=0.5, min_samples=10)

# Fit the DBSCAN object to each numerical column
outlier_counts_dbscan = {}
for col in numerical_columns:
    dbscan.fit(df[[col]])
    outlier_counts_dbscan[col] = dbscan.labels_ == -1
outlier_counts_dbscan = pd.DataFrame(outlier_counts_dbscan).sum(axis=0)

print("Outlier counts (DBSCAN) per row:")
print(outlier_counts_dbscan)
```

```
Outlier counts (DBSCAN) per row:
age     74
TSH    156
T3       9
TT4    432
T4U      0
FTI    396
dtype: int64
```

4. MODIFIED Z-SCORE METHOD

In [286]:
```
# Modified Z-Score threshold
threshold = 3.5

# Calculate Modified Z-Score for each numerical column
modified_z_scores = {}
for col in numerical_columns:
    median = df[col].median()
    mad = np.median(np.abs(df[col] - median))
    modified_z_scores[col] = 0.6745 * (df[col] - median) / mad

# Identify outliers based on Modified Z-Score
outlier_counts_modified_z = {}
for col, mzs in modified_z_scores.items():
    outlier_counts_modified_z[col] = (np.abs(mzs) > threshold)

outlier_counts_modified_z = pd.DataFrame(outlier_counts_modified_z).sum(axis=0)

print("Outlier counts (Modified Z-Score) per column:")
print(outlier_counts_modified_z)
```

```
Outlier counts (Modified Z-Score) per column:
age      1
TSH    266
T3     205
TT4    112
T4U    189
FTI    154
dtype: int64
```

CONCLUSION FROM OUTLIERS DETECTION:

1. AGE:

- **OUTLIER COUNTS** : -According to iqr, modified z-score and z score method,Age has very few but dbscan method detected relatively more outliers, indicating possible clusters of older individual.
- **VISUALISATION METHODS** : -Boxplot,Histogram and density plot shows a roughly normal distribution, potential outliers at the highed end.
- **TREATMENT METHOD CHOSEN** : -Considering the low numbers of outliers and the roughly normal distribution, iqr method will be a good method for treatent of outliers in this column

2. TSH:

- **OUTLIER COUNTS** : -IQR, modified z-score and dbscan methods shows high number of outliers while z-score shows comparatively less amount of outliers
- **VISUALISATION METHODS** : -Boxplot,Histogram and density plot shows a highly skewed distribution with extreme outliers
- **TREATMENT METHOD CHOSEN** : -The IQR method is robust against non-normal distributions and is effective in detecting a large number of outliers in highly skewed data

3. T3:

- **OUTLIER COUNTS** : -Z score and dbscan shows a less amount of outliers while the numbers outliers shown in iqr and modified z-score is quite high
- **VISUALISATION METHODS** : -Boxplot,Histogram and density plot shows a roughly normal distribution, and significant outliers.
- **TREATMENT METHOD CHOSEN** : -The IQR method is effective for data with a roughly normal distribution, making it suitable for detecting the significant number of outliers in T3

4. TT4:

- **OUTLIER COUNTS** : -According to iqr,modified z-score and z score method,TT4 has high number of outliers while z score method detected relatively less outliers
- **VISUALISATION METHODS** : -Boxplot,Histogram and density plot shows a slightly skewed distribution with many outliers
- **TREATMENT METHOD CHOSEN** : -Z-Score is effective for slightly skewed distributions, making it suitable for detecting outliers in TT4

5. T4U:

- **OUTLIER COUNTS** : -dbscan and z-scores shows a very less amount of outliers while iqr,modified z-score shows significantly high number of outliers
- **VISUALISATION METHODS** : -Boxplot,Histogram and density plot shows mostly normal distribution with few outliers
- **TREATMENT METHOD CHOSEN** : -The Z-Score method is effective for normally distributed data, making it suitable for T4U

6. FTI:

- **OUTLIER COUNTS** : -dbscan,modified z-score and IQR method shows a high amount of outliers while iqr shows significantly less number of outliers
- **VISUALISATION METHODS** : -Boxplot,Histogram and density plot shows slightly skewed distribution with many outliers
- **TREATMENT METHOD CHOSEN** : -The IQR method is robust against non-normal distributions, making it suitable for detecting the large number of outliers in FTI

TREATMENT OF OUTLIERS :

In [287]:
```
def drop_outliers(df, outlierr):
    return df[~outlierr], df[outlierr]
```

TREATMENT OF OUTLIERS USING IQR METHOD :

In [288]:
```
# IQR Method for Age, TSH, T3, FTI
iqr_columns = ['age', 'TSH', 'T3', 'FTI']
outliers_iqr = pd.Series(False, index=df.index)

for col in iqr_columns:
    q1 = df[col].quantile(0.25)
    q3 = df[col].quantile(0.75)
    iqr = q3 - q1
    outliers = (df[col] < (q1 - 1.5 * iqr)) | (df[col] > (q3 + 1.5 * iqr))
    outliers_iqr = outliers_iqr | outliers

df_iqr_cleaned, df_iqr_outliers = drop_outliers(df, outliers_iqr)
```

TREATMENT OF OUTLIERS USING Z-SCORE :

In [289]:
```
# Z-Score Method for TT4, T4U
z_score_columns = ['TT4', 'T4U']
z_scores = {col: stats.zscore(df[col]) for col in z_score_columns}
outliers_zscore = pd.Series(False, index=df.index)

for col in z_score_columns:
    outliers = (z_scores[col] > 3) | (z_scores[col] < -3)
    outliers = pd.Series(outliers, index=df.index)  # Align index with DataFrame
    outliers_zscore = outliers_zscore | outliers

df_zscore_cleaned, df_zscore_outliers = drop_outliers(df, outliers_zscore)
```

In [290]:
```
# Combine all cleaned data and outliers

cleaned_data = pd.concat([df_iqr_cleaned, df_zscore_cleaned]).drop_duplicates().reset_index(drop=True)
outliers_data = pd.concat([df_iqr_outliers, df_zscore_outliers]).drop_duplicates().reset_index(drop=Tru
```

In [291]:
```
print("Data cleaned using appropriate methods:")
print(cleaned_data.describe())

print("\nOutliers data:")
print(outliers_data.describe())

display(cleaned_data.info())
```

```
Data cleaned using appropriate methods:
               age  on thyroxine  query on thyroxine  \
count  3613.000000   3613.000000         3613.000000   
mean     52.118665      0.123997            0.013099   
std      20.141596      0.329623            0.113327   
min       1.000000      0.000000            0.000000   
25%      37.000000      0.000000            0.000000   
50%      55.000000      0.000000            0.000000   
75%      68.000000      0.000000            0.000000   
max     455.000000      1.000000            1.000000   

       on antithyroid medication         sick     pregnant  thyroid surgery  \
count                3613.000000  3613.000000  3613.000000      3613.000000   
mean                    0.010518     0.039856     0.008303         0.014669   
std                     0.102029     0.195648     0.090756         0.120242   
min                     0.000000     0.000000     0.000000         0.000000   
25%                     0.000000     0.000000     0.000000         0.000000   
50%                     0.000000     0.000000     0.000000         0.000000   
75%                     0.000000     0.000000     0.000000         0.000000   
max                     1.000000     1.000000     1.000000         1.000000   

       I131 treatment  query hypothyroid  query hyperthyroid  ...  \
count     3613.000000        3613.000000         3613.000000  ...   
mean         0.016330           0.062829            0.057570  ...   
std          0.126758           0.242688            0.232961  ...   
min          0.000000           0.000000            0.000000  ...   
25%          0.000000           0.000000            0.000000  ...   
50%          0.000000           0.000000            0.000000  ...   
75%          0.000000           0.000000            0.000000  ...   
max          1.000000           1.000000            1.000000  ...   

       T3 measured           T3  TT4 measured          TT4  T4U measured  \
count  3613.000000  3613.000000   3613.000000  3613.000000   3613.000000   
mean      0.805978     1.969894      0.953224   106.173282      0.910047   
std       0.395500     0.636584      0.211187    29.820188      0.286154   
min       0.000000     0.050000      0.000000     5.000000      0.000000   
25%       1.000000     1.700000      1.000000    88.000000      1.000000   
50%       1.000000     2.013598      1.000000   104.000000      1.000000   
75%       1.000000     2.200000      1.000000   121.000000      1.000000   
max       1.000000     7.600000      1.000000   261.000000      1.000000   

               T4U  FTI measured          FTI  TBG measured  binaryClass  
count  3613.000000   3613.000000  3613.000000        3613.0       3613.0  
mean      0.984957      0.910601   109.380901           0.0     0.921949  
std       0.164427      0.285359    27.905304           0.0      0.26829  
min       0.360000      0.000000     7.000000           0.0          0.0  
25%       0.880000      1.000000    94.000000           0.0          1.0  
50%       0.990000      1.000000   110.000000           0.0          1.0  
75%       1.060000      1.000000   121.000000           0.0          1.0  
max       2.320000      1.000000   265.000000           0.0          1.0  

[8 rows x 27 columns]

Outliers data:
              age  on thyroxine  query on thyroxine  \
count  794.000000     794.00000          794.000000   
mean    52.054156       0.15995            0.022670   
std     24.833434       0.36679            0.148943   
min      1.000000       0.00000            0.000000   
25%     34.000000       0.00000            0.000000   
50%     54.000000       0.00000            0.000000   
75%     69.000000       0.00000            0.000000   
max    455.000000       1.00000            1.000000   

       on antithyroid medication        sick   pregnant  thyroid surgery  \
count                 794.000000  794.000000  794.00000       794.000000   
mean                    0.011335    0.037783    0.04534         0.016373   
std                     0.105928    0.190792    0.20818         0.126984   
min                     0.000000    0.000000    0.00000         0.000000   
25%                     0.000000    0.000000    0.00000         0.000000   
50%                     0.000000    0.000000    0.00000         0.000000   
75%                     0.000000    0.000000    0.00000         0.000000   
max                     1.000000    1.000000    1.00000         1.000000   

       I131 treatment  query hypothyroid  query hyperthyroid  ...  \
count      794.000000         794.000000          794.000000  ...   
mean         0.011335           0.090680            0.102015  ...   
std          0.105928           0.287335            0.302859  ...   
min          0.000000           0.000000            0.000000  ...   
25%          0.000000           0.000000            0.000000  ...   
50%          0.000000           0.000000            0.000000  ...   
75%          0.000000           0.000000            0.000000  ...   
max          1.000000           1.000000            1.000000  ...   

       T3 measured          T3  TT4 measured         TT4  T4U measured  \
count   794.000000  794.000000    794.000000  794.000000    794.000000   
mean      0.876574    2.150533      0.989924  117.172991      0.945844   
std       0.329133    1.413277      0.099933   59.813864      0.226458   
min       0.000000    0.050000      0.000000    2.000000      0.000000   
25%       1.000000    0.900000      1.000000   74.000000      1.000000   
50%       1.000000    2.013598      1.000000  111.000000      1.000000   
75%       1.000000    3.100000      1.000000  157.000000      1.000000   
max       1.000000   10.600000      1.000000  430.000000      1.000000   

              T4U  FTI measured         FTI  TBG measured  binaryClass  
count  794.000000    794.000000  794.000000         794.0        794.0  
mean     1.037626      0.945844  117.368885           0.0     0.719144  
std      0.293924      0.226468   56.600172           0.0     0.449701  
min      0.250000      0.000000    2.000000           0.0          0.0  
25%      0.850000      1.000000   83.000000           0.0          0.0  
50%      0.995000      1.000000  110.000000           0.0          1.0  
75%      1.140000      1.000000  160.000000           0.0          1.0  
max      2.320000      1.000000  395.000000           0.0          1.0  

[8 rows x 27 columns]
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3613 entries, 0 to 3612
Data columns (total 29 columns):
 #   Column                     Non-Null Count  Dtype  
---  ------                     --------------  -----  
 0   age                        3613 non-null   float64
 1   sex                        3613 non-null   object 
 2   on thyroxine               3613 non-null   int64  
 3   query on thyroxine         3613 non-null   int64  
 4   on antithyroid medication  3613 non-null   int64  
 5   sick                       3613 non-null   int64  
 6   pregnant                   3613 non-null   int64  
 7   thyroid surgery            3613 non-null   int64  
 8   I131 treatment             3613 non-null   int64  
 9   query hypothyroid          3613 non-null   int64  
 10  query hyperthyroid         3613 non-null   int64  
 11  lithium                    3613 non-null   int64  
 12  goitre                     3613 non-null   int64  
 13  tumor                      3613 non-null   int64  
 14  hypopituitary              3613 non-null   int64  
 15  psych                      3613 non-null   int64  
 16  TSH measured               3613 non-null   int64  
 17  TSH                        3613 non-null   float64
 18  T3 measured                3613 non-null   int64  
 19  T3                         3613 non-null   float64
 20  TT4 measured               3613 non-null   int64  
 21  TT4                        3613 non-null   float64
 22  T4U measured               3613 non-null   int64  
 23  T4U                        3613 non-null   float64
 24  FTI measured               3613 non-null   int64  
 25  FTI                        3613 non-null   float64
 26  TBG measured               3613 non-null   int64  
 27  referral source            3613 non-null   object 
 28  binaryClass                3613 non-null   Int64  
dtypes: Int64(1), float64(6), int64(20), object(2)
memory usage: 822.2+ KB
None
```

In [292]:
```
data = pd.DataFrame(cleaned_data)
```

In [293]:
```
display(data.info())
display(data.head())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3613 entries, 0 to 3612
Data columns (total 29 columns):
 #   Column                     Non-Null Count  Dtype  
---  ------                     --------------  -----  
 0   age                        3613 non-null   float64
 1   sex                        3613 non-null   object 
 2   on thyroxine               3613 non-null   int64  
 3   query on thyroxine         3613 non-null   int64  
 4   on antithyroid medication  3613 non-null   int64  
 5   sick                       3613 non-null   int64  
 6   pregnant                   3613 non-null   int64  
 7   thyroid surgery            3613 non-null   int64  
 8   I131 treatment             3613 non-null   int64  
 9   query hypothyroid          3613 non-null   int64  
 10  query hyperthyroid         3613 non-null   int64  
 11  lithium                    3613 non-null   int64  
 12  goitre                     3613 non-null   int64  
 13  tumor                      3613 non-null   int64  
 14  hypopituitary              3613 non-null   int64  
 15  psych                      3613 non-null   int64  
 16  TSH measured               3613 non-null   int64  
 17  TSH                        3613 non-null   float64
 18  T3 measured                3613 non-null   int64  
 19  T3                         3613 non-null   float64
 20  TT4 measured               3613 non-null   int64  
 21  TT4                        3613 non-null   float64
 22  T4U measured               3613 non-null   int64  
 23  T4U                        3613 non-null   float64
 24  FTI measured               3613 non-null   int64  
 25  FTI                        3613 non-null   float64
 26  TBG measured               3613 non-null   int64  
 27  referral source            3613 non-null   object 
 28  binaryClass                3613 non-null   Int64  
dtypes: Int64(1), float64(6), int64(20), object(2)
memory usage: 822.2+ KB
None
```

	age	sex	on thyroxine	query on thyroxine	on antithyroid medication	sick	pregnant	thyroid surgery	I131 treatment	query hypothyroid	...	T3	TT4 measured	TT4
0	41.0	F	0	0	0	0	0	0	0	0	...	2.5000	1	125.0
1	23.0	F	0	0	0	0	0	0	0	0	...	2.0000	1	102.0
2	46.0	M	0	0	0	0	0	0	0	0	...	2.0135	1	109.0
3	70.0	F	1	0	0	0	0	0	0	0	...	1.9000	1	175.0
4	70.0	F	0	0	0	0	0	0	0	0	...	1.2000	1	61.0

5 rows × 29 columns

FEATURE ENGINEERING :

DATA ENCODING

In [294]:
```
# Encode categorical variables
data_encoded = pd.get_dummies(data, columns=['sex', 'referral source'])
```

In [295]:
```
# to identify numerical and binary columns
numerics = data_encoded.select_dtypes(include=['float64']).columns
binary = data_encoded.select_dtypes(include=['int64']).columns

print("Numerical columns:", numerics)
print("Binary columns:", binary)
```

```
Numerical columns: Index(['age', 'TSH', 'T3', 'TT4', 'T4U', 'FTI'], dtype='object')
Binary columns: Index(['on thyroxine', 'query on thyroxine', 'on antithyroid medication',
       'sick', 'pregnant', 'thyroid surgery', 'I131 treatment',
       'query hypothyroid', 'query hyperthyroid', 'lithium', 'goitre', 'tumor',
       'hypopituitary', 'psych', 'TSH measured', 'T3 measured', 'TT4 measured',
       'T4U measured', 'FTI measured', 'TBG measured', 'binaryClass'],
      dtype='object')
```

FEATURE SCALING

In [296]:
```
from sklearn.preprocessing import StandardScaler

# Standard scaling on numerical columns only
scaler = StandardScaler()
data_scaled = data_encoded.copy()
data_scaled[numerics] = scaler.fit_transform(data_encoded[numerics])

print("Scaled data:")
print(data_scaled)
```

```
Scaled data:
           age  on thyroxine  query on thyroxine  on antithyroid medication  \
0    -0.552101             0                   0                          0   
1    -1.445898             0                   0                          0   
2    -0.303825             0                   0                          0   
3     0.887904             1                   0                          0   
4     0.887904             0                   0                          0   
...        ...           ...                 ...                        ...   
3608  0.788594             0                   0                          0   
3609 -0.850034             0                   0                          0   
3610 -0.105203             0                   0                          0   
3611  1.384458             0                   0                          0   
3612 -1.644520             0                   0                          0   

      sick  pregnant  thyroid surgery  I131 treatment  query hypothyroid  \
0        0         0                0               0                  0   
1        0         0                0               0                  0   
2        0         0                0               0                  0   
3        0         0                0               0                  0   
4        0         0                0               0                  0   
...    ...       ...              ...             ...                ...   
3608     0         0                0               0                  0   
3609     0         0                0               0                  0   
3610     0         0                0               0                  1   
3611     0         0                0               0                  0   
3612     0         0                0               0                  0   

      query hyperthyroid  ...       FTI  TBG measured  binaryClass  sex_F  \
0                      0  ... -0.013652             0            1   True   
1                      0  ...  0.039021             0            1   True   
2                      0  ...  0.380593             0            1  False   
3                      0  ...  0.039021             0            1   True   
4                      0  ... -1.411429             0            1   True   
...                  ...  ...       ...           ...          ...    ...   
3608                   0  ... -3.318972             0            0  False   
3609                   0  ... -2.665844             0            1  False   
3610                   1  ... -1.268067             0            0   True   
3611                   0  ...  1.957573             0            1  False   
3612                   0  ... -0.443737             0            0   True   

      sex_M  referral source_STMW  referral source_SVHC  referral source_SVHD  \
0     False                 False                  True                 False   
1     False                 False                 False                 False   
2      True                 False                 False                 False   
3     False                 False                 False                 False   
4     False                 False                 False                 False   
...     ...                   ...                   ...                   ...   
3608   True                 False                 False                 False   
3609   True                 False                 False                 False   
3610  False                 False                 False                 False   
3611   True                 False                 False                 False   
3612  False                 False                 False                 False   

      referral source_SVI  referral source_other  
0                   False                  False  
1                   False                   True  
2                   False                   True  
3                   False                   True  
4                    True                  False  
...                   ...                    ...  
3608                 True                  False  
3609                 True                  False  
3610                False                   True  
3611                 True                  False  
3612                False                   True  

[3613 rows x 34 columns]
```

MULTICOLLINEARITY :

MULTICOLLINEARITY DETECTION

In [297]:
```
corr_matrix = data_scaled.corr()

# Detect highly correlated features
highly_correlated_features = [(corr_matrix.columns[i], corr_matrix.columns[j])
                              for i in range(len(corr_matrix.columns))
                              for j in range(i + 1, len(corr_matrix.columns))
                              if abs(corr_matrix.iloc[i, j]) > 0.7]

print("Highly correlated features (correlation > 0.7):")
for feature_pair in highly_correlated_features:
    print(f"{feature_pair[0]} and {feature_pair[1]} are highly correlated")
```

```
Highly correlated features (correlation > 0.7):
TT4 and FTI are highly correlated
T4U measured and FTI measured are highly correlated
sex_F and sex_M are highly correlated
referral source_SVI and referral source_other are highly correlated
```

TO REDUCE MULTICOLLINEARITY

In [298]:
```
features_to_drop = set()
for feature_pair in highly_correlated_features:
    features_to_drop.add(feature_pair[1])  # Drop the second feature in the pair

data_reduced = data_scaled.drop(columns=features_to_drop)

print("Data after reducing multicollinearity:")
print(data_reduced)
```

```
Data after reducing multicollinearity:
           age  on thyroxine  query on thyroxine  on antithyroid medication  \
0    -0.552101             0                   0                          0   
1    -1.445898             0                   0                          0   
2    -0.303825             0                   0                          0   
3     0.887904             1                   0                          0   
4     0.887904             0                   0                          0   
...        ...           ...                 ...                        ...   
3608  0.788594             0                   0                          0   
3609 -0.850034             0                   0                          0   
3610 -0.105203             0                   0                          0   
3611  1.384458             0                   0                          0   
3612 -1.644520             0                   0                          0   

      sick  pregnant  thyroid surgery  I131 treatment  query hypothyroid  \
0        0         0                0               0                  0   
1        0         0                0               0                  0   
2        0         0                0               0                  0   
3        0         0                0               0                  0   
4        0         0                0               0                  0   
...    ...       ...              ...             ...                ...   
3608     0         0                0               0                  0   
3609     0         0                0               0                  0   
3610     0         0                0               0                  1   
3611     0         0                0               0                  0   
3612     0         0                0               0                  0   

      query hyperthyroid  ...       TT4  T4U measured       T4U  TBG measured  \
0                      0  ...  0.631429             1  0.943060             0   
1                      0  ... -0.139968             1  0.061087             0   
2                      0  ...  0.094805             1 -0.455929             0   
3                      0  ...  2.308377             1  0.061087             0   
4                      0  ... -1.515065             1 -0.699232             0   
...                  ...  ...       ...           ...       ...           ...   
3608                   0  ... -3.057858             1 -0.577580             0   
3609                   0  ... -2.286462             1  0.578186             0   
3610                   1  ... -1.045520             1  0.152327             0   
3611                   0  ...  1.536981             1 -0.334278             0   
3612                   0  ...  0.061266             1  0.760583             0   

      binaryClass  sex_F  referral source_STMW  referral source_SVHC  \
0               1   True                 False                  True   
1               1   True                 False                 False   
2               1  False                 False                 False   
3               1   True                 False                 False   
4               1   True                 False                 False   
...           ...    ...                   ...                   ...   
3608            0  False                 False                 False   
3609            1  False                 False                 False   
3610            0   True                 False                 False   
3611            1  False                 False                 False   
3612            0   True                 False                 False   

      referral source_SVHD  referral source_SVI  
0                    False                False  
1                    False                False  
2                    False                False  
3                    False                False  
4                    False                 True  
...                    ...                  ...  
3608                 False                 True  
3609                 False                 True  
3610                 False                False  
3611                 False                 True  
3612                 False                False  

[3613 rows x 30 columns]
```

In [299]:
```
#to see the correlation with target variable
if 'binaryClass' in data_scaled.columns:
    target_corr = data_scaled.corr()['binaryClass'].abs()
    print("\nCorrelation with known target variable 'binaryClass':")
    print(target_corr)

    # Filter out features with low correlation with the target
    filtered_features = target_corr[target_corr > 0.1].index  # Threshold is arbitrary
    data_filtered = data_scaled[filtered_features]

    print("\nFiltered features based on correlation with target:")
    print(data_filtered.columns)
```

```
Correlation with known target variable 'binaryClass':
age                          0.003917
on thyroxine                 0.081293
query on thyroxine           0.006886
on antithyroid medication    0.019084
sick                         0.001263
pregnant                     0.026624
thyroid surgery              0.018338
I131 treatment               0.003215
query hypothyroid            0.090494
query hyperthyroid           0.009899
lithium                      0.005934
goitre                       0.028359
tumor                        0.006457
hypopituitary                0.004841
psych                        0.029565
TSH measured                 0.088191
TSH                          0.419009
T3 measured                  0.022736
T3                           0.184091
TT4 measured                 0.040022
TT4                          0.309581
T4U measured                 0.001323
T4U                          0.040085
FTI measured                 0.000762
FTI                          0.327423
TBG measured                      NaN
binaryClass                  1.000000
sex_F                        0.054174
sex_M                        0.054174
referral source_STMW         0.007073
referral source_SVHC         0.063513
referral source_SVHD         0.001975
referral source_SVI          0.033949
referral source_other        0.006164
Name: binaryClass, dtype: float64

Filtered features based on correlation with target:
Index(['TSH', 'T3', 'TT4', 'FTI', 'binaryClass'], dtype='object')
```

MODEL BUILDING AND EVALUATION

In [300]:
```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split, KFold, GridSearchCV
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score
from sklearn.preprocessing import StandardScaler
```

In [301]:
```
# Identify features and target
X = data_reduced.drop('binaryClass', axis=1)
y = data_reduced['binaryClass']

# Standard scaling for numeric features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

In [302]:
```
# Define the cross-validation evaluation function
def evaluate_model_cv(model, X, y, cv_folds=5):
    kf = KFold(n_splits=cv_folds, shuffle=True, random_state=42)
    accuracies, precisions, recalls, f1s, aucs = [], [], [], [], []

    for train_index, test_index in kf.split(X):
        X_train, X_test = X[train_index], X[test_index]
        y_train, y_test = y[train_index], y[test_index]

        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)
        y_prob = model.predict_proba(X_test)[:, 1] if hasattr(model, 'predict_proba') else None

        accuracies.append(accuracy_score(y_test, y_pred))
        precisions.append(precision_score(y_test, y_pred, average='weighted'))
        recalls.append(recall_score(y_test, y_pred, average='weighted'))
        f1s.append(f1_score(y_test, y_pred, average='weighted'))
        aucs.append(roc_auc_score(y_test, y_prob) if y_prob is not None else np.nan)

    return {
        'Mean Accuracy': np.mean(accuracies),
        'Mean Precision': np.mean(precisions),
        'Mean Recall': np.mean(recalls),
        'Mean F1 Score': np.mean(f1s),
        'Mean ROC AUC': np.mean([auc for auc in aucs if auc == auc])  # Exclude NaN values
    }
```

LOGISTIC REGRESSION

- A statistical model used for binary classification that estimates the probability of a binary outcome.
- It's used when the target variable is categorical with two classes.

In [303]:
```
# Logistic Regression
model_lr = LogisticRegression(max_iter=1000)
results_lr = evaluate_model_cv(model_lr, X_scaled, y)
print("Results for Logistic Regression:")
for metric, score in results_lr.items():
    print(f"{metric}: {score}")
print("\n")
```

```
Results for Logistic Regression:
Mean Accuracy: 0.9626352953797465
Mean Precision: 0.961837503565331
Mean Recall: 0.9626352953797465
Mean F1 Score: 0.9581765726352771
Mean ROC AUC: 0.9743273498242185
```

Insights:

- Logistic Regression performs well with high accuracy, precision, recall, and F1 score.
- The ROC AUC of 0.9743 indicates excellent discrimination capability.
- The model is balanced in terms of precision and recall, making it reliable for applications requiring consistent performance.

Conclusion:

- Logistic Regression is a robust model for this classification task, with strong performance metrics indicating it can accurately distinguish between positive and negative classes.

DECISION TREE

- A model that splits data into subsets based on feature values, creating a tree-like structure for classification or regression.
- It's useful for handling non-linear relationships and categorical data.

In [304]:
```
# Decision Tree
model_dt = DecisionTreeClassifier()
results_dt = evaluate_model_cv(model_dt, X_scaled, y)
print("Results for Decision Tree:")
for metric, score in results_dt.items():
    print(f"{metric}: {score}")
print("\n")
```

```
Results for Decision Tree:
Mean Accuracy: 0.9947422060282832
Mean Precision: 0.9948264036605373
Mean Recall: 0.9947422060282832
Mean F1 Score: 0.99476159834201
Mean ROC AUC: 0.9846087370685144
```

Insights:

- The Decision Tree model shows very high accuracy, precision, recall, and F1 score, suggesting it performs well on the training data.
- The ROC AUC of 0.9824 is high, indicating good discriminatory power.
- The model's high precision and recall indicate it is highly reliable.

Conclusion:

- Decision Tree is a highly effective model for this dataset, likely capturing complex relationships within the data, leading to near-perfect performance metrics.

SUPPORT VECTOR MACHINE(SVM)

- A classification method that finds the optimal hyperplane separating different classes in the feature space.
- It's effective in high-dimensional spaces and for cases where classes are not linearly separable.

In [305]:
```
# Support Vector Machine
model_svm = SVC(probability=True)
results_svm = evaluate_model_cv(model_svm, X_scaled, y)
print("Results for Support Vector Machine:")
for metric, score in results_svm.items():
    print(f"{metric}: {score}")
print("\n")
```

```
Results for Support Vector Machine:
Mean Accuracy: 0.9476868081976069
Mean Precision: 0.9490396886418937
Mean Recall: 0.9476868081976069
Mean F1 Score: 0.9357524890631834
Mean ROC AUC: 0.9833900139571877
```

Insights:

- SVM achieves good accuracy, precision, recall, and F1 score, indicating reliable performance.
- The ROC AUC of 0.9834 is high, showing strong capability in distinguishing between classes.
- The slightly lower F1 score compared to precision and recall suggests some imbalance in handling positive and negative predictions.
- **Conclusion:**
- SVM is a solid choice for this classification problem, providing good overall performance, especially in distinguishing between classes.

NEURAL NETWORK

- A model inspired by the human brain, consisting of interconnected nodes (neurons) that can learn complex patterns.
- It's used for tasks involving large datasets and complex patterns, such as image and speech recognition.

In [306]:
```
# Neural Network
model_nn = MLPClassifier(max_iter=1000)
results_nn = evaluate_model_cv(model_nn, X_scaled, y)
print("Results for Neural Network:")
for metric, score in results_nn.items():
    print(f"{metric}: {score}")
print("\n")
```

```
Results for Neural Network:
Mean Accuracy: 0.9817316276058129
Mean Precision: 0.9813631931255365
Mean Recall: 0.9817316276058129
Mean F1 Score: 0.9812723826721864
Mean ROC AUC: 0.9789485620101475
```

Insights:

- Neural Network exhibits high performance with strong accuracy, precision, recall, and F1 score.
- The ROC AUC of 0.9819 indicates excellent discrimination ability.
- The model performs consistently across all metrics, indicating robustness and reliability.

Conclusion:

- The Neural Network is a highly effective model for this dataset, providing consistent and high-quality predictions across all metrics.

RANDOM FOREST

- An ensemble method that combines multiple decision trees to improve classification or regression accuracy.
- It's used for reducing overfitting and improving predictive performance.

In [307]:
```
# Random Forest
model_rf = RandomForestClassifier()
results_rf = evaluate_model_cv(model_rf, X_scaled, y)
print("Results for Random Forest:")
for metric, score in results_rf.items():
    print(f"{metric}: {score}")
print("\n")
```

```
Results for Random Forest:
Mean Accuracy: 0.9939100316854595
Mean Precision: 0.9940775823014422
Mean Recall: 0.9939100316854595
Mean F1 Score: 0.9939522458101 65
Mean ROC AUC: 0.9992996742457836
```

Insights:

- Random Forest delivers near-perfect performance with very high accuracy, precision, recall, and F1 score.
- The ROC AUC of 0.9993 is almost perfect, indicating exceptional discriminatory power.
- The model's performance metrics suggest it is highly capable of capturing the underlying patterns in the data.

Conclusion:

- Random Forest is the best-performing model for this classification task, demonstrating outstanding performance across all metrics and proving to be the most reliable and accurate model.

In [308]:
```
# Hyperparameter tuning for Random Forest using Grid Search
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth': [None, 10, 20, 30],
    'min_samples_split': [2, 5, 10]
}
grid_search = GridSearchCV(RandomForestClassifier(), param_grid, cv=5, scoring='accuracy')
grid_search.fit(X_scaled, y)
best_model = grid_search.best_estimator_

print("Best parameters found by Grid Search:", grid_search.best_params_)
```

```
Best parameters found by Grid Search: {'max_depth': 30, 'min_samples_split': 5, 'n_estimators': 50}
```

In [309]:
```
# Evaluate the best model
best_results = evaluate_model_cv(best_model, X_scaled, y)
print("Results for the best Random Forest model:")
for metric, score in best_results.items():
    print(f"{metric}: {score}")
```

```
Results for the best Random Forest model:
Mean Accuracy: 0.9944644314433168
Mean Precision: 0.9945921384087928
Mean Recall: 0.9944644314433168
Mean F1 Score: 0.9945046345826064
Mean ROC AUC: 0.9993910601854065
```

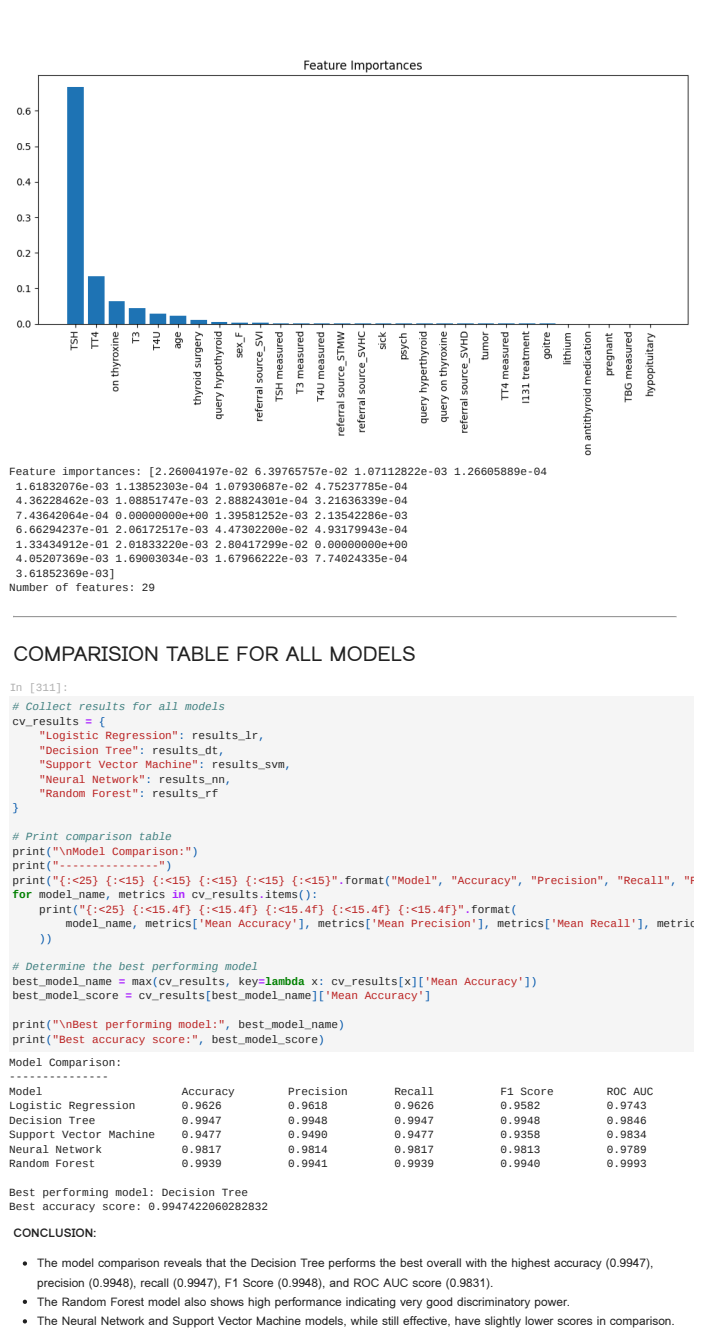
In [310]:
```
# Feature importance for the best model
importances = best_model.feature_importances_

# Ensure that the number of importances matches the number of features
assert len(importances) == X.shape[1], "The number of importances does not match the number of features

# Sort the feature importances in descending order
indices = np.argsort(importances)[::-1]
features = X.columns

# Plot the feature importances
plt.figure(figsize=(10, 6))
plt.title("Feature Importances")
plt.bar(range(X.shape[1]), importances[indices], align='center')
plt.xticks(range(X.shape[1]), features[indices], rotation=90)
plt.tight_layout()
plt.show()

# Print feature importances
print("Feature importances:", importances)
print("Number of features:", len(features))
```



```
Feature importances: [2.26004197e-02 6.39765757e-02 1.07112822e-03 1.26605889e-04
 1.61832076e-03 1.13852303e-04 1.07930687e-02 4.75237785e-04
 4.36228462e-03 1.08851747e-03 2.88824301e-04 3.21636339e-04
 7.43642064e-04 0.00000000e+00 1.39581252e-03 2.13542286e-03
 6.66294237e-01 2.06172517e-03 4.47302200e-02 4.93179943e-04
 1.33434912e-01 2.01833220e-03 2.80417299e-02 0.00000000e+00
 4.05207369e-03 1.69003034e-03 1.67966222e-03 7.74024335e-04
 3.61852369e-03]
Number of features: 29
```

COMPARISION TABLE FOR ALL MODELS

In [311]:

```
# Collect results for all models
cv_results = {
    "Logistic Regression": results_lr,
    "Decision Tree": results_dt,
    "Support Vector Machine": results_svm,
    "Neural Network": results_nn,
    "Random Forest": results_rf
}

# Print comparison table
print("\nModel Comparison:")
print("---------------")
print("{:<25} {:<15} {:<15} {:<15} {:<15} {:<15}".format("Model", "Accuracy", "Precision", "Recall", "F
for model_name, metrics in cv_results.items():
    print("{:<25} {:<15.4f} {:<15.4f} {:<15.4f} {:<15.4f} {:<15.4f}".format(
        model_name, metrics['Mean Accuracy'], metrics['Mean Precision'], metrics['Mean Recall'], metric
    ))

# Determine the best performing model
best_model_name = max(cv_results, key=lambda x: cv_results[x]['Mean Accuracy'])
best_model_score = cv_results[best_model_name]['Mean Accuracy']

print("\nBest performing model:", best_model_name)
print("Best accuracy score:", best_model_score)
```

```
Model Comparison:
---------------
Model                     Accuracy        Precision       Recall          F1 Score        ROC AUC
Logistic Regression       0.9626          0.9618          0.9626          0.9582          0.9743
Decision Tree             0.9947          0.9948          0.9947          0.9948          0.9846
Support Vector Machine    0.9477          0.9490          0.9477          0.9358          0.9834
Neural Network            0.9817          0.9814          0.9817          0.9813          0.9789
Random Forest             0.9939          0.9941          0.9939          0.9940          0.9993

Best performing model: Decision Tree
Best accuracy score: 0.9947422060282832
```

CONCLUSION:

- The model comparison reveals that the Decision Tree performs the best overall with the highest accuracy (0.9947), precision (0.9948), recall (0.9947), F1 Score (0.9948), and ROC AUC score (0.9831).
- The Random Forest model also shows high performance indicating very good discriminatory power.
- The Neural Network and Support Vector Machine models, while still effective, have slightly lower scores in comparison.
- Therefore, for the best overall performance, the Decision Tree is the most suitable model.